Ders 7 Genişletilmiş Notu: Swap Yöntemleri, Pisagor Üçlüsü, Floyd Üçgeni, Dizi Ters Çevirme ve Palindromlar

Ders 7 Genişletilmiş Notu: Swap Yöntemleri, Pisagor Üçlüsü, Floyd Üçgeni, Dizi Ters Çevirme ve Palindromlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Derse Giriş: Algoritmik Yapılar Neden Yetmiyor?
- 2. İki Değişkenin Yer Değiştirmesi (Swap)
- 3. Geçici Değişkensiz Swap Yöntemleri
- 4. Toplamı 1000 Olan Pisagor Üçlüsü
- 5. Floyd Üçgeni: n. Satırın Son Elemanı
- 6. n Elemanlı Dizinin Ters Çevrilmesi
- 7. Palindrom Kontrolü
- 8. 6 Basamaklı Palindrom Sayılar
- 9. R Basamaklı Kaç Palindrom Sayı Vardır?
- 10. Uygulama Soruları
- 11. Kod Standartları ve Kısıt Altında Çözüm
- 12. Ders Sonrası Soru: Kosinüs Serisi

1. Derse Giriş: Algoritmik Yapılar Neden Yetmiyor?

Bu ders, uygulama saatleriyle birleştirilmiş uzun bir derstir. Hoca, diğer grupta asistanların çözdüğü uygulama sorularının bu grupta kendisi tarafından çözüleceğini, ana soruların ise Fatih Hoca'nın ders notlarından alınacağını söyledi. Fatih Hoca'nın sayfasındaki çalışma sorularının her hafta güncellendiğini ve iyi alıştırmalar olduğunu da ekledi.

Hoca önce geçen dersi hatırlattı. Son derste döngüler üzerine çalışılmış, örneğin puana göre sıralama yapmak istendiğinde yalnızca algoritmik yapıların yetersiz kaldığı görülmüştü. Yani sadece algoritmayla, başka bir yapı kullanmadan her problemi çözmek mümkün değildir, başka şeylere de ihtiyaç vardır. Diziler bu yüzden gerekir. Hoca, BBG dersini almamış öğrenciler olduğunu da düşünerek bu konuları "böyle bir sorunla karşılaşınca ne yaparız?" diye problem üzerinden anlatmayı daha doğru bulduğunu söyledi.

2. İki Değişkenin Yer Değiştirmesi (Swap)

2.1 Swap Nerede Karşımıza Çıkar?

Hoca, Fatih Hoca'nın ders notundaki ilk konudan, yani iki değişken arasında swap işleminden başladı ve bilgisayarda swap'ın nerede karşımıza çıktığını sordu.

Bir öğrenci kitaplık örneğini verdi: Düzenli bir kitaplığa araya yeni bir kitap koymak istediğimizde diğerlerini kaydırmamız gerekir. Hoca bunu kabul etti. Bir düzen vardır ve araya eleman eklerken bazı elemanları yedeklemek gerekir.

Soru: İki banka hesabı arasındaki para transferi bir swap örneği olabilir mi? **Cevap:** Hayır. Para gönderildiğinde karşı hesap yalnızca artar, karşılıklı bir değiş tokuş olmaz. "Sen bana gönderince ben sana bir şey göndermiyorum."

Bir başka öğrencinin verdiği örnek sıralama algoritmalarındaki karşılaştırmaydı. Hoca asıl örneğin bu olduğunu söyledi. Sıralama bu dersin önemli bir konusu olacak ve 3-4 farklı algoritma görülecektir. Bellekte bir dizide duran elemanları sıralarken bir elemanı başka bir elemanla yer değiştirmek gerekir. Yer değiştirirken de geçici (temporary) olarak kullanılacak yeni bir yere ihtiyaç vardır.

Swap edilecek şeyin türü de çok çeşitli olabilir: Boolean bir değişken, karakter, string, ileride görülecek nesne yönelimli programlamadaki bir nesne veya bir pointer.

2.2 Makine Seviyesinde Exchange Komutu

Hoca, en temel seviyeden yani sıfır ve birler seviyesinden başladı. Bazı işlemcilerde `exchange` veya `swap` adlı bir makine komutu vardır. Bu komut iki bellek hücresinin veya iki register'ın içeriğini tek bir seferde yer değiştirir. Arka planda yine bir algoritma çalışır, ama bu algoritma mantık kapıları seviyesindedir.

Programlama dillerinde böyle bir lüksümüz yoktur. En azından C'de `exchange` komutu tanımlı değildir. Kendimiz bir swap fonksiyonu yazabiliriz, ama henüz fonksiyon yazmadığımız için buna gerek de yoktur.

2.3 Geçici Değişkenle Swap

Elimizde `x` ve `y` adlı iki değişken varsa işlem şöyle yapılır:

1. `x`, geçici değişkene kopyalanır. Artık `x`'in değeri iki yerde birden durur.
2. `y`, `x`'e kopyalanır.
3. Geçici değişkendeki değer `y`'ye atanır.

```
temp = x;
x = y;
y = temp;
```

Hoca derste örnek olarak $x = 2$, $y = 4$ değerlerini verdi. Hangi değişkenle başladığımız önemli değildir. `y`'yi önce geçici değişkene alsaydık, sonra `x`'i `y`'ye atardık.

> **Hocanın vurgusu:** Öğrenciler bunu yazarken nedense karıştırıyor, herhalde yeterince kod yazmamaktan kaynaklanıyor. Kuralı şöyle düşünebilirsiniz: `temp`'ler dışta kalır, `x` ve `y` birbirine bakar.

Soru: `temp`'e `x`'i atadığımızda `x`'teki değer boşalıyor mu? **Cevap:** Hayır. Eşittir (assignment) operatörü sağdaki veriyi alır, soldakinin içine kopyalar. Veri iki yerde birden olur, yer değiştirmez. Hoca buna ileride "deep copy" diyeceğimizi söyledi: Aynı veri klonlanır.

2.4 Daha Önce Gördüğümüz Benzer Yedeklemeler

Hoca, bu yedekleme mantığının geçen cuma dersinde iki yerde kullanıldığını hatırlattı.

- **Fibonacci:** `x1`, `x2` ve `x3` vardı. `x3`, `x1 + x2` olarak hesaplanıyordu. Bir sonraki eleman için önce `x2`, `x1`'e atanıyor, sonra güncel `x3`, `x2`'ye alınıyordu. Burada tam bir swap yok, ama bir yedekleme söz konusudur.
- **İkinci en büyük:** Daha büyük bir değer gelince önce birinci en büyük, ikinci en büyüğe yedekleniyor, sonra birinci en büyük kendini güncelliyordu.

2.5 Üç Değişkenli Döndürme

Hoca soruyu biraz değiştirdi: `x` değeri `y`'ye, `y` değeri `z`'ye, `z` değeri de `x`'e atanmak isteniyor. Bir öğrenci önce toplamayı önerdi, hoca bunun şu anda karıştırılmaması gerektiğini söyledi. Başka bir öğrencinin çözümü istenen çözümdü:

```
temp = z;
z = y;
y = x;
x = temp;
```

Burada da `temp` yine dışta kaldı, çapraz olanlar birbirine saklandı. Bu mantık farklı farklı durumlarda kullanılır.

3. Geçici Değişkensiz Swap Yöntemleri

Hoca, geçici değişken kullanmadan da yer değiştirme yapılabileceğini söyledi ve bunun için öneri istedi.

Bir öğrenci x'i döngüyle y'ye eşit olana kadar artırıp sonra yerlerini değiştirmeyi önerdi. Hoca, bunun için önce hangisinin büyük olduğunu bulmak gerektiğini, bunun da fazla uğraştıracağını, `temp` kullanmanın daha kolay olduğunu söyledi.

3.1 Toplama ve Çıkarma ile

```
x = x + y;
y = x - y;
x = x - y;
```

Hoca değerler yerine sembollerle izledi:

Satır	x'in değeri	y'nin değeri
Başlangıç	x	y
`x = x + y`	$x + y$	y
`y = x - y`	$x + y$	$(x + y) - y = x$
`x = x - y`	$(x + y) - x = y$	x

Sonuçta y'de ilk x, x'te ilk y kalır.

Soru: Program yazarken yalnız iki değişken kullanıyoruz, ama bilgisayar $x + y$ değerini hesaplarken bir yerde saklamak zorunda değil mi? Yine yer kullanmıyor mu? **Cevap:** Bu bir operasyondur. Önce toplama işlemi yapılır, x, y ve toplama komutu işlemciye gider, ardından atama yapılır. Değer büyük ihtimalle işlemcinin içinde saklanır veya program koşarken bellekte geçici bir hücre oluşturulur. Hoca, nerede saklandığını bilmediğini, işin o tarafını çok bilmediğini açıkça söyledi. Ayrıca tek bir komut çevriminde iki register'ı toplayıp sonucu yine birine yazan işlemciler de vardır: x yerine R1, y yerine R2 kullanılır, "bununla bunu topla, buna ata" denir.

Soru (sohbetten): Birden fazla `temp` kullanılabilir mi, `temp` tek değer mi alır? **Cevap:** `temp` de bir değişkendir ve aynı türden değişkenler üzerinde çalışıyoruz. Bütün değişkenler aynı anda tek değer alır, ekstra bir durum yoktur.

3.2 Çarpma ve Bölme ile

Hoca aynı düşüncenin çarpma ve bölmeyle de çalıştığını gösterdi:

```
x = x * y;
y = x / y;
x = x / y;
```

Satır	x'in değeri	y'nin değeri
`x = x * y`	xy	y
`y = x / y`	xy	$xy/y = x$
`x = x / y`	$xy/x = y$	x

3.3 ^ Üs Değildir

Sohbette bir öğrenci üs alma işlemiyle (x üzeri y) yapmayı düşündüğünü, ama çıkamadığını yazdı. Hoca, üssün tersinin logaritma olduğunu, o zaman logaritmaya girmek gerektiğini söyledi. Asıl yöntem başka bir öğrencinin sohbete yazdığı XOR'dur.

> **Hocanın vurgusu:** C'de ^ operatörünün adı XOR'dur, eksponent değildir.

3.4 XOR ile

XOR iki girişli bir mantıksal işlemdir ve binary sistemde doğrudan karşılığı vardır. Girişler aynıysa çıkış 0, farklıysa çıkış 1 olur.

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Hoca, XOR'un kriptolojide çok kullanıldığını söyledi. Genelde bir anahtar ve bir mesaj vardır. Anahtar ile mesaj bit bit XOR işlemine tabi tutulur, çıkan sonuç şifreli mesajdır. Hocanın örneğinde mesaj `101`, anahtar `110`'dur. Sağdan sola: $1 \oplus 0 = 1$, $0 \oplus 1 = 1$, $1 \oplus 1 = 0$ (son adım ve tablo slayttan alındı). Şifreli mesaj `011` olur.

Swap için yine aynı kalıp kullanılır:

```
x = x ^ y;
y = x ^ y;
x = x ^ y;
```

Hocanın örneği $x = 2$, $y = 4$'tür. İkilikte dört bit açılırsa, geri kalan bütün bitler 0'dır:

Adım	İşlem	Sonuç
Başlangıç		`x = 0010`, `y = 0100`
1	`x = x ^ y` = `0010 ^ 0100`	`x = 0110`
2	`y = x ^ y` = `0110 ^ 0100`	`y = 0010` (ilk x)
3	`x = x ^ y` = `0110 ^ 0010`	`x = 0100` (ilk y)

Böylece iki değer yer değiştirir. Sonuç Python ile denendi: `x = 4`, `y = 2`.

3.5 Yöntemlerin Karşılaştırılması: Bir Trade-off

Hoca bütün yöntemleri bellek ve işlem sayısıyla karşılaştırdı.

Yöntem	Ek bellek	Atama	Ek işlem
Geçici değişkenle	+1	3	yok
Toplama-çıkarma	0	3	+3 toplama/çıkarma
Çarpma-bölme	0	3	+3 çarpma/bölme
XOR	0	3	+3 XOR

Burada toplama ile çıkarma aynı işlem gibi sayılır. Yine bir trade-off ile karşılaştık: `temp` kullanılırsa yalnızca atama yapılır ama bir bellek alanı gerekir. `temp` kullanılmazsa bellek gerekmez, ama 3 atamaya ek olarak 3 işlem daha yapılır. Tercih probleme ve üzerinde çalışılan donanıma göre yapılır. Bir bellek hücresi kadar bile yeriniz yoksa `temp`'siz yöntemler tercih edilebilir.

İşin bir de doğruluk tarafı vardır. `temp` kullanınca tip uyumsuzluğu problemi olmaz, çünkü `temp`, `x` ve `y` ile aynı türden tanımlanır. `temp` kullanılmadığında ise bir değerin üzerine başka bir

değer eklenir. Bu değerin bir üst limiti vardır ve limit aşılırsa kayıp olur. Hocanın örneği şudur: 4 byte'lık `int` değişkenlerin ikisi de 2 milyar olsun. Toplamları 4 milyar olması gerekirken `int` sınırını aşar. Çarpmada da aynı sorun vardır. Bölmede ayrıca ikinci sayı sıfırsa sıfıra bölme (division by zero) oluşabilir.

> **Ders sonrası not:** Hoca `int` üst limitini sözlü olarak "2 milyar 127 diye gidiyordu" dedi. Doğrusu, 4 byte'lık `int` için $2^{31} - 1 = 2\,147\,483\,647$'dir (Python ile hesaplandı).
>
> **Hocanın vurgusu:** Doğruluk (correctness) bizim için önemlidir. Buradaki yöntemlerde algoritmanın doğruluğundan kayıp olabilir, buna dikkat etmek gerekir.

Hoca XOR için "bu problemler olmayabilir" diye düşündüğünü, ama ara sonuçların sınırı aşıp aşmadığını çok incelemediğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Hocanın açık bıraktığı soru için: XOR bit bit çalıştığından büyük olasılıkla taşma üretmez, sıfırla da sorun çıkarmaz. İki değişkenin değeri eşit olsa bile swap doğru sonuçlanır (`gcc` ile denendi). Sorun, iki ifadenin aynı bellek hücresini göstermesidir: Örneğin `A[i]` ile `A[j]` için $i = j$ olursa ilk adımda değer sıfırlanır ve kaybolur (bu da `gcc` ile denendi). Ayrıca `^` büyük olasılıkla yalnızca tam sayı tiplerinde kullanılabilir.
>
> **Ek bilgi (derste anlatılmadı):** Toplama-çıkarma yöntemindeki işaretli tam sayı taşması büyük olasılıkla C standardına göre tanımsız davranıştır (undefined behavior). Pratikte değer çoğu derleyicide "sarılır" ve swap yine doğru çıkabilir, ama buna güvenilmemelidir.

4. Toplamı 1000 Olan Pisagor Üçlüsü

4.1 Soru

Hoca, önce akış diyagramı çizilecek, sonra kodu yazılacak bir soru verdi. Pozitif tam sayılar için şu iki şart aynı anda sağlanmalıdır:

$$a + b + c = 1000, \qquad a^2 + b^2 = c^2, \qquad a, b, c > 0$$

a, b, c ne olabilir? Hoca, Fatih Hoca'nın notundaki soruda büyüklük sırasıyla ilgili bir şart daha bulunduğunu fark etti ve onu ek çalışma sorusu olarak bıraktı: Bulunan a, b, c bir şarttan daha geçirilecektir.

Düşünme süresinin başında hoca genel bir ilke hatırlattı: Üç yapı kullanıyoruz (sıralı, şartlı, tekrarlı) ve mümkün oldukça tekrarlı yapıdan kaçınmamız gerekir.

4.2 Birinci Yöntem: Üç İç İçe Döngü

Bir öğrenci önce işlemle başlamak isteyince hoca, burada algoritma ve akış diyagramı istendiğini söyledi. "İç içe for açtım" demenin de yetmediğini, bilgisayarın bunu bilmediğini, adım adım tarif edilmesi gerektiğini belirtti. Öğrencinin kurduğu akış şöyledir:

1. `a`, 1'den 999'a birer birer artan bir döngüdür.
2. Bu döngünün her adımında `b`, 1'den 999'a gider.
3. Onun her adımında `c`, 1'den 999'a gider.
4. En içte bir şart kontrol edilir. Doğruysa `a, b, c` yazdırılır.
5. Yanlışsa en içteki döngünün sonraki adımına geçilir. İç döngü bitince ortadaki, o bitince dıştaki döngüye dönülür.
6. Dış döngü bitince program durur.

Öğrencinin şartı ilginçti. Alttaki denklemi düzenleyerek $(1000 - a)(c - b) = a^2$ yazmış, `t = (1000 - a) * (c - b)` hesaplayıp `t == a * a` olup olmadığına bakmıştı. Burada $1000 - a$'nın parantez içinde olması gerekir.

Hoca akışı ilk değerlerle izledi. $a = b = c = 1$ için $t = 999 \times 0 = 0$ olur ve $a^2 = 1$'e eşit değildir. $c = 2$ için $t = 999 \times 1 = 999$ olur, yine eşit değildir. Hoca bu eşitliğin ayrıca doğrulanması gerektiğini söyleyip soru işareti bıraktı.

Soru: Bu döngülerin içine kaç defa girilir? **Cevap:** Yaklaşık bir milyar. $1000 \times 1000 \times 1000 = 10^9$ defa.

Hoca, bu kadar döneceksek çok matematiksel bir şeye girmeye gerek olmadığını söyledi ve iki şartı doğrudan test etti: `a*a + b*b == c*c` ve `a + b + c == 1000`. Program çalışınca şunları yazdırdı:

```
200 375 425
375 200 425
```

Program bütün olası durumları denediği için aynı çözüm `a` ile `b` yer değiştirmiş olarak iki kez bulunur. Bir öğrenci bunu fark etti: a önce 200, sonra 375 bulunuyor. Başka bir öğrenci, `b`'nin üst sınırı olarak 999 yerine `a` kullanılabileceğini önerdi, hoca da "unutma, geleceğiz" dedi.

Hoca, $a > b > c$ şartı da verilseydi yalnızca ilk satırın çıkacağını, bunun için ekstra bir `if` yazılabileceğini söyledi.

> **Ders sonrası not:** Hoca bu ek şartı $a > b > c$ diye söyledi. Bu şartla çözüm olamaz, çünkü $a^2+b^2 = c^2$ ise c en büyük olmak zorundadır. Hocanın "ilk satırı verir" dediği sonuca (200, 375, 425) karşılık gelen şart $a < b < c$'dir. Fatih Hoca'nın notundaki şart da büyük olasılıkla budur; not altyazıda okunmuyor.

4.3 Dönüştürülmüş Formülün Denenmesi

Hoca aynı döngüleri öğrencinin formülüyle de çalıştırdı. Çok daha fazla sonuç geldi ve "bir kere 600 ve 895'i toplayınca 1000 şartını burada ihlal eden bir durum var" dedi, üzerinde çalışılmasını önerdi.

Formül aslında doğru türetilmiştir: $a^2 = c^2 - b^2 = (c-b)(c+b)$ yazılır ve $c + b = 1000 - a$ kullanılır (ara adım eklendi). Fakat bu türetmede $a + b + c = 1000$ kullanıldığı için, `c` ayrı bir döngüyle serbestçe seçilince toplam şartı kendiliğinden sağlanmaz.

> **Ders sonrası not:** Python ile denendi: Bu döngülerde yalnız $(1000 - a)(c - b) = a^2$ şartını sağlayan 2321 üçlü vardır. Bunlardan yalnızca ikisinin toplamı 1000'dir (200, 375, 425 ve 375, 200, 425). Yani formül kullanılacaksa toplam şartı ayrıca kontrol edilmelidir. Hocanın söylediği 600 ve 895 bu 2321 üçlünün hiçbirinde birlikte geçmiyor; büyük olasılıkla ekrandaki farklı satırlardaki değerlerdir.

4.4 İkinci Yöntem: `c` Döngüsünü Kaldırmak

Hoca bir milyar turda boşa giden işe dikkat çekti: a 500 iken b 600'lere, 700'lere gidiyor. Bunlar zaten 1000 sınırını aşar. Daha önemlisi, `c` için döngüye hiç gerek yoktur. Toplam 1000 olacağına göre:

$$c = 1000 - a - b$$

Bir döngü azaldı, tur sayısı $1000 \times 1000 = 10^6$'ya düştü. Yine de bir sorun kalır: a 500 iken `b`'nin 501 olması gereksizdir. Gereksiz döngü, gereksiz toplama-çıkarma ve gereksiz `if` kontrolü yapılır. Bu yüzden `b`'nin sınırı `a`'ya bağlanır:

$$b < 1000 - a$$

Hoca sınırları tek tek inceledi. `b`, $1000 - a$ değerine ulaşırsa `c` sıfır olur. `a`'nın üst sınırı 999 alınırsa ($a < 1000$) $a = 999$, $b = 1$ yine `c`'yi sıfırlar. Bu yüzden `a` için $a < 999$ yazılır, yani `a` en fazla 998 olur. O zaman `b` en fazla 1 olur, `c` de 1 olur. Bütün değerler pozitif kalır.

```
for (a = 1; a < 999; a++)
    for (b = 1; b < 1000 - a; b++) {
        c = 1000 - a - b;
        if (a*a + b*b == c*c)
            printf("%d %d %d\n", a, b, c);
    }
```

Tur sayısı artık 10^6'dan da küçüktür. Hoca, ileride bunun için $n(n+1)/2$ gibi bir formül kuracağımızı söyledi. Sayaçla ölçülen değer de buna uyar: $\frac{998\times999}{2} = 498\,501$.

Soru: Neden `while` ile yazmadık? **Cevap:** Bütün durumları denemek durumundayız. Aradığımız şey 1000'e kadar olan değerlerin hepsinde olabilir, bir başlangıç ve bitiş limitimiz vardır. `while` ile de yazılabilir, ama `for` bu iş için uygun bir döngüdür. Döngünün arada kesilmesi gibi bir durum yoktur: Program 200, 375, 425'i bulunca yazar ve aramaya devam eder. Soruda "ilk gördüğünde çık" denseydi `while` yapılabilirdi.

Hoca iki kodu art arda çalıştırınca ilk döngünün bitmesini gözle beklemek gerektiğini, ikinci döngünün ise hemen bittiğini gösterdi. Çünkü 10^9 büyük bir sayıdır ve aradaki fark kayda değerdir.

Şartları `&&` ile bağlarken her birini paranteze almak iyi olur. Hoca, `&&` operatörünün önceliğinin bazen hatırlanamayabildiğini, garantici olmanın bazı durumlarda iyi olduğunu söyledi.

4.5 Üçüncü Yöntem: Sınırları Daha da Daraltmak

Bir öğrenci `a`'nın 500'den küçük olması gerektiğini söyledi: $a^2 + b^2 = c^2$ olduğundan c hem a'dan hem b'den büyüktür. $a = 500$ olsaydı c en az 501 olur ve toplam 1000'i geçerdi. Hoca önce itiraz etti, örneğin $a = b = 1$ iken c'nin 998 olabileceğini ve bir büyüklük ilişkisi görmediğini söyledi. Öğrenci gerekçeyi açıklayınca kabul etti. Hoca bunu daha önce düşünmediğini ve öğrenciler arasında böyle fikirlerin çıkmasının güzel olduğunu söyledi. Bunları tek tek kodla denemek gerektiğini de ekledi.

Aradan hemen önce başka bir öğrenci fikri tamamladı. $a+b$ toplamı c'den büyük olmak zorundadır. Toplam 1000 olduğuna göre $a+b > 500$ demektir. Bu yüzden `b`, $500-a$'dan başlatılabilir. `b`'nin üst sınırı da 500 alınabilir, çünkü toplamı 1000 olan bir üçlüde b, c'den küçüktür. Hoca bunu üçüncü metot olarak koda ekledi:

```
for (a = 1; a < 500; a++)
    for (b = 500 - a; b < 500; b++) {
        c = 1000 - a - b;
        if (a*a + b*b == c*c)
            printf("%d %d %d\n", a, b, c);
    }
```

Kod sayaç değişkeniyle birlikte çalıştırıldı. Her `a` için tam `a` tur döner, toplam $1+2+\cdots+499 = 124\,750$ tur olur (ara adım eklendi).

4.6 Dördüncü Yöntem: `do-while` ile

Bir öğrenci, `c`'nin `a`'dan ve `b`'den büyük olma şartını `while` ile kontrol etmeyi önerdi ve kodunu sohbetten attı: `b`, bu şart sağlandığı sürece artırılır, böylece bütün değerlere bakmak gerekmez. Hoca fikri beğendi.

Hoca öğrencilerle kodu sadeleştirdi. `1000 - a - b` üç yerde ayrı ayrı hesaplanıyordu. `c` bir kez hesaplanıp şartta `c` kullanılırsa işlem azalır. Fakat `while` girişinde `c` henüz bilinmediği için `c`'yi döngüden önce bir kez daha hesaplamak gerekir. Bir öğrenci `do-while`'ı sordu. Hoca, en az bir defa çalışacağı için `do-while`'a çevirmenin daha güzel olacağını söyledi. Hoca kodu yazarken `do-while`'ın sonundaki noktalı virgülü unuttu ve derleme hatasını görünce ekledi.

```
for (a = 1; a < 999; a++) {
    b = 1;
    do {
        c = 1000 - a - b;
        if (a*a + b*b == c*c)
            printf("%d %d %d\n", a, b, c);
        b++;
    } while (c >= a && c >= b);
}
```

Ders sonrası not: Altyazıda şart "`c`, `a`'dan ve `b`'den büyük" olarak geçiyor, kodun o satırı karede görünmüyor. `gcc` ile denendi: `c > a && c > b` ile sayaç 166.832 çıkıyor.

Ekrandaki 167.165 değerini `c >= a && c >= b` veriyor, bu yüzden derste yazılan şart büyük olasılıkla budur. Her iki biçim de aynı iki çözümü buluyor.

4.7 Sayaçların Karşılaştırılması

Hoca dört yöntemi bir `count` değişkeniyle aynı programda çalıştırdı. Ekran çıktısı (kareden):

```
200 375 425
375 200 425
Sayac=997002999
Ikinci method
200 375 425
375 200 425
Sayac=498501
Ucuncu method
200 375 425
375 200 425
Sayac=124750
 200 375 425
 375 200 425
Sayac=167165
```

Yöntem	Döngü sınırları	Tur sayısı	Hesap
1	`a, b, c`: 1..999	997.002.999	999^3
2	`a` < 999, `b` < 1000 − `a`	498.501	$998 \cdot 999/2$
3	`a` < 500, `b`: 500 − `a`..499	124.750	$499 \cdot 500/2$
4	`a` < 999, `b`: 1'den, `do-while`	167.165	ölçüldü

Bu sayılar ve çözümler ders sonrasında `gcc -Wall -std=c89` ile aynı biçimde elde edildi. Dört yöntemin de sonucu aynıdır.

Birincinin verimsizliği gözle görülür. Dördüncünün üçüncüden daha fazla dönmesi hocanın ilgisini çekti. Hoca bunu, dördüncü yöntemde `b`'nin 1'den başlayıp daha üst değerlere de gitmesiyle açıkladı. Üçüncü yöntemde ise `b` belli bir aralıkla sınırlandırılmıştır. Son `b` değerini ekrana yazdırınca 2 gördü, bu da son dış döngü adımındaki değerdir. Hoca bu döngüyü doğrudan analiz edemediğimizi söyledi. Üçüncü yöntemin sınırlarının da özel olduğunu belirtti: 1000 değil de 10 bin deseydik yine ona göre düşünmek gerekirdi.

Özet olarak, ilk akla gelen yöntemden başlayıp farklı düşüncelerle daha ileri noktaya gidildi. Hoca bunun üzerinde çalışmaya değer olduğunu söyledi.

5. Floyd Üçgeni: n. Satırın Son Elemanı

Hoca bu soruyu da döngüyle düşünmek üzere verdi. Sayılar satırlara şöyle dizilir:

```
1
2 3
4 5 6
7 8 9 10
...
```

Soru şudur: 1000. satırın son elemanı kaçtır? Hoca üçgenin adını Pascal üçgeni sanarak söyledi, bir öğrenci Floyd üçgeni olduğunu hatırlattı.

5.1 Birinci Çözüm: İç İçe İki Döngü

Bilgisayarcı gözüyle düşünürsek birinci satırda sayaç bir kez, ikinci satırda iki kez, üçüncü satırda üç kez artar. O zaman iki döngü vardır:

```
t = 0;
for (i = 1; i <= n; i++)
    for (j = 1; j <= i; j++)
        t = t + 1;
```

İşlem sayısı hesaplanırsa iç döngü i defa, dış döngü n defa döner:

$$\sum_{i=1}^{n}\sum_{j=1}^{i} 1 = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{n^2+n}{2}$$

Hoca slayta 1000. satır için sonucu 500500 olarak yazdı. Python ile doğrulandı: $\frac{1000\times1001}{2} = 500\,500$.

5.2 İkinci Çözüm: Tek Döngü

Bir öğrenci tek döngüyle bulunabileceğini önerdi. Her satırın son elemanı, bir önceki satırın son elemanına satır numarası eklenerek bulunur:

```
t = 1;
for (i = 2; i <= n; i++)
    t = t + i;
```

Hoca bunu denedi:

- $n = 3$: $t = 1$, $i = 2$ ile $t = 3$, $i = 3$ ile $t = 6$. Doğru.
- $n = 4$: 1, 3, 6, sonra $i = 4$ ile $t = 10$. Doğru.

Burada döngü 2'den n'e gittiği için $n - 1$ işlem yapılır. Adım sayısı n^2'li bir ifadeden n'li bir ifadeye düşer.

5.3 Üçüncü Çözüm: Formül

Başka bir öğrenci son sayının bir toplam olduğunu söyledi. Birinci satırda bir, ikinci satırda iki, n. satırda n tane değer vardır. O zaman n. satırın son elemanı:

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Bu ifade 3 işlemde hesaplanır: bir toplama, bir çarpma, bir bölme (işlemlerin dökümü eklendi).

Hoca üç çözümü yan yana koydu. Hepsi aynı değeri bulur, ama birinci çözümde $(n^2 + n)/2$, ikincide $n-1$, üçüncüde 3 işlem yapılır. 1000. satır için bu sayılar 500.500, 999 ve 3'tür (Python ile hesaplandı).

> **Ders sonrası not:** Hoca karşılaştırırken birinci çözüm için kısaca "n^2+n tane" dedi, doğrusu $(n^2 + n)/2$'dir. Altyazıda n değeri 100, işlem sayısı "100.000'den fazla" ve "99 defa" olarak duyuluyor. Bu sayılar birbirini tutmuyor: $n = 100$ için doğrusu 5050 ve 99, $n = 1000$ için 500.500 ve 999'dur. Slayttaki soru 1000. satır olduğundan hoca büyük olasılıkla $n = 1000$'i kastetti.
>
> **Hocanın vurgusu:** Algoritma geliştirmek iteratif bir süreçtir. Bu soruda o süreci yaşamaya çalıştık.

6. n Elemanlı Dizinin Ters Çevrilmesi

6.1 Hangi Eleman Hangisiyle Yer Değiştirir?

Hoca önce 8 elemanlı bir dizi yazdı. Ters çevirince 1. ile 8., 2. ile 7., 3. ile 6., en son 4. ile 5. eleman yer değiştirir. İndis olarak düşünürsek ilk çift 0 ile 7'dir.

7 elemanlı dizide 1 ile 7, 2 ile 6, 3 ile 5 yer değiştirir. Ortadaki 4 olduğu gibi kalır.

6.2 Akış Diyagramı

Hoca, daha önce konuşmamış bir öğrenciden akış diyagramını istedi. İndisler 1'den başlatılarak şu adımlar kuruldu:

1. Başla.
2. `n` değerini kullanıcıdan al.
3. `i` 1'den `n`'e kadar bir döngüyle `A[i]` elemanlarını al.
4. `i` 1'den $\lfloor n/2 \rfloor$'ye kadar birer birer artan bir döngü aç.
5. Döngü içinde: `temp = A[i]`, `A[i] = A[n-i+1]`, `A[n-i+1] = temp`.
6. `i` 1'den `n`'e kadar `A[i]` elemanlarını yazdır.
7. Dur.

Hoca bir başka öğrenciye şunu sordu: Döngüde $n/2$ yerine n deseydik ne olurdu? **Cevap:** $n/2$'ye kadar elemanlar yer değiştirir, sonra bir kez daha değiştirilir ve eski hâllerine dönerler. Dizi orijinal hâline geri gelir.

6.3 C Kodu

Kaç eleman gireceğimizi bilmiyoruz. Hoca, program çalışırken işletim sisteminden belli bir yer istemenin farklı bir fonksiyonla yapıldığını ve bunu sonra göreceğimizi söyledi. Şimdilik `#define` ile büyük bir maksimum tanımlanır, kullanıcıdan alınan `n` kadarı kullanılır.

```c
#include <stdio.h>
#define MAX 100
int main() {
    int i, n, t;
    int A[MAX];

    printf("Eleman sayisi giriniz");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("%d . eleman=", i + 1);
        scanf("%d", &A[i]);
    }
    for (i = 0; i < n / 2; i++) {
        t = A[i];
        A[i] = A[n - i - 1];
        A[n - i - 1] = t;
    }
    printf("Dizinin son hali\n");
    for (i = 0; i < n; i++)
        printf("%d ", A[i]);
    return 0;
}
```

`scanf` satırında `&` kullanmak zorundayız, çünkü `A`'nın `i`. hücresinin adresini söylemeliyiz. Kullanmazsak hücredeki değer alınır ve oraya bir şey yazamayız.

Akış diyagramında `A[n-i+1]` yazılırken C'de `A[n-i-1]` yazılmasının sebebini hoca da yazarken karıştırıp düzeltti. Akış diyagramında `i` 1'den başlar, son eleman da `n` numaralıdır. C'de `i` 0'dan başlar, son elemanın indisi de `n - 1`'dir.

> **Hocanın vurgusu:** C'de kod yazıyorsanız diziler sıfırdan başlar. Döngüler istediğiniz yerden başlar, ama dizide `i` sıfırdan başlamak zorundadır. "Kafam karışıyor, 1'den başlatayım, boyutu da MAX + 1 yapayım" demeyin, buna alışacaksınız. Sıfırıncı elemanı boşuna yazmıyoruz.

Hoca, `n/2` yerine bazı sistemlerde geçici bir değişken kullanmak gerektiğini de söyledi. Bir öğrencinin önerisi üzerine `while` ile de olabileceğini, ama birinci elemandan $n/2$. elemana kadar gidileceği için `for`'un yeterli olduğunu belirtti.

Hoca önce 6 elemanla (1, 2, 3, 4, 5, 6) denedi, sonuç 6 5 4 3 2 1 çıktı. Sonra 5 elemanla (1, 2, 3, 4, 5) denedi, sonuç 5 4 3 2 1 çıktı. Kod ders sonrasında `gcc -Wall -std=c89` ile derlendi,

aynı sonuçları verdi.

6.4 `n/2` ve Tek Sayıda Eleman

Hoca özellikle 5 elemanlı örneği seçti, çünkü `n / 2` bir tam sayı bölmesidir. Algoritma düzeyindeki normal bölmenin sonucu tam sayı değildir. Bu yüzden aşağıya yuvarlama düşünülmelidir ve akış diyagramında $\lfloor n/2 \rfloor$ yazılması daha doğrudur. 7 eleman varsa 3,5'tan 3'e iniyoruz: Ortadaki elemana dokunmuyoruz, çünkü onun yer değiştireceği başka eleman yoktur.

Dokunulsa eleman kendisiyle yer değiştirir ve boşuna iş yapılır. Hoca bu durumu slayta "if n is odd, then changing the middle point leads to unstable condition" diye yazdı. Sıralama algoritmalarının "stable" olup olmamasıyla ilişkilendirdi ve Wikipedia'daki sıralama karşılaştırma tablosunda böyle bir sütun olduğunu gösterdi.

> **Ders sonrası not:** Hoca, ortadaki elemanın kendisiyle yer değiştirmesini "unstable" sözcüğüyle anlattı. Sıralama algoritmalarındaki kararlılık (stability) ise büyük olasılıkla eşit değerli elemanların sıralamadan sonra birbirlerine göre ilk sıralarını korumasıdır. Bir elemanın kendisiyle yer değiştirmesi yalnızca gereksiz bir işlemdir, kararlılıkla ilgisi yoktur.

Soru: Swap'tan önce `if` ile iki değerin aynı olup olmadığına baksak, aynıysa hiç değiştirmesek olur mu? **Cevap:** Lokasyonlar farklı ama değerler aynı olabilir, öneri bunu kastediyor. Fakat bu kontrol için de `A[i]` ve `A[n-i-1]` erişimleri, toplama-çıkarma ve == işlemleri yapılır. Hoca bunu 5-6 işlem olarak saydı. Aynı erişimler swap içinde zaten yapılacaktır. Değiştirme çoğu durumda olacağı için kontrol gereksiz iş ekler. Tekrar etme olasılığı yüksek değerlerin olduğu durumlarda kullanılabilir, ama bunu da istatistiksel olarak açıklamak gerekir.

6.5 `int A[n]` Olur mu?

Soru: Başta `MAX` tanımladık. Onun yerine `n` girildikten sonra doğrudan `A[n]` tanımlasak olur muydu? **Cevap:** Hoca C'de bunun böyle olmadığını, bunun dinamik bellek konusuna girdiğini söyledi. Program önce çalışacak, sonra dizi için yer ayıracak demektir. İşletim sistemi bazen izin verir, bazen hata verir. Bunu derste denedi: 500 eleman çalıştı, biraz daha büyük değer de çalıştı, çok büyük değerde "segmentation fault" alındı. Her zaman verecek diye bir garanti yoktur. Bu iş pointer'larla başka türlü yapılacaktır. Şimdilik büyük bir maksimum tanımlanır, aşağıda daha küçük bir `n` ile çalışılır.

> **Ders sonrası not:** Hoca "C'de bu böyle olmuyor" dedi. Bu yazım büyük olasılıkla C99'da değişken uzunluklu dizi (VLA) olarak vardır, derste kullanılan C89'da ise yoktur. Denendi: `gcc -std=c89 -pedantic` uyarı veriyor ama derliyor, küçük değerlerde çalışıyor, çok büyük değerde burada da "segmentation fault" alınıyor. Hata büyük olasılıkla yerel dizinin stack'e sığmamasından kaynaklanır. Bu derste `#define MAX` ile sabit boyutlu dizi kullanılmalıdır.

6.6 Dizinin Sınırı Dışına Erişim

Soru: Bazen fark etmeden döngüyü bir fazla, `i <= n` diye yazıyorum ve bir değer okuyorum. Başka bir programın belleğini okumuş olabilir miyim? **Cevap:** Başka bir programın address space'ine gidemezsin, onu işletim sistemi engeller. Sana ayrılan proses alanı içinde, programdaki ayrılan yerin dışındaki bir yere erişirsin. Yine sana ayrılan bir yerdesin, ama istediğinden daha fazlasını kullanıyorsun.

Bu yüzden genelde `i < n` yazılır, `i <= n` yazılmaz. `i` hiçbir zaman `n`'e ulaşmaz. Bu hatayı yapan öğrenciler bazen "sıfır gördüm" der. Sıfır görmek normaldir, ama bazen çok büyük sayılar da görülür. Çünkü C bellekteki çöp değerleri sıfırlamaz, sizin ilklendirmenizi bekler.

> **Ders sonrası not:** Hoca bu proses alanı için "process control block" dedi. Bu ad büyük olasılıkla işletim sisteminin proses hakkında tuttuğu bilgi yapısıdır, prosesin adres alanı değildir. Ayrıca sınır dışına taşan her erişimin "yine sana ayrılan yerde" kalması da büyük olasılıkla garanti değildir: Taşma büyükse program "segmentation fault" ile sonlanabilir.

Hocanın vurgusu: Dizilerde döngü şartı `i < n` veya `i < n/2` olur. Bu hata zaman zaman yapılıyor, önemli bir durumdur.

7. Palindrom Kontrolü

Fatih Hoca'nın sorusu şudur: Bir dizi veya string palindrom mudur? Palindrom, tersi ve kendisi birbirine eşit olan dizi veya stringtir. Sayı dizisi de karakter dizisi de olabilir. Örneğin `abba` palindromdur, `abbb` değildir. 1441 palindromdur, 1444 değildir.

7.1 Naif Çözüm: İkinci Bir Dizi

İlk ve en basit yöntem bazı kitaplarda da böyle anlatılır. Dizi yeni bir diziye ters kopyalanır. Yeni diziye kopyalarken yer değiştirme gerekmez, doğrudan atama yapılır:

```
B[i] = A[n - i - 1];
```

`i` sıfırdan başlar. `B[0]`'a `A[n-1]`, `B[1]`'e `A[n-2]` gelir ve böyle devam eder. Sonra iki dizi baştan karşılaştırılır.

Soru: Bu karşılaştırma `for` ile mi, `while` ile mi yapılmalı? **Cevap (bir öğrenci):** `while` ile, eşit olmadığını gördüğü anda çıkması için. Hoca bu cevabı onayladı. Döngü bittiğinde `i` son elemana kadar gelmişse palindrom, gelmemişse palindrom değil yazdırılır.

```c
#include <stdio.h>
#define MAX 100
int main() {
    int i, n;
    int A[MAX], B[MAX];

    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &A[i]);
    for (i = 0; i < n; i++)
        B[i] = A[n - i - 1];
    i = 0;
    while (i < n - 1 && B[i] == A[i])
        i++;
    if (i == n - 1)
        printf("palindrom\n");
    else
        printf("palindrom degil\n");
    return 0;
}
```

Ders sonrası not: Hoca bu çözümü anlatırken iki yerde kısa geçti. Birincisi, ters kopyalama için "$n/2$'ye kadar gideceğiz" dedi. Bu sınır yerinde ters çevirme içindir. Yeni diziye kopyalarken döngü bütün n eleman için dönmelidir, yukarıdaki kod da öyle yazıldı. İkincisi, `while` şartını yalnız `B[i] == A[i]` olarak söyledi, ama bu şart dizi palindromsa `i`'yi dizinin dışına taşırır. Bu yüzden koda `i < n - 1` eklendi. Kod `gcc -Wall -std=c89` ile derlendi, 1 4 4 1 ve 1 2 3 2 1 için "palindrom", 1 2 3 1 için "palindrom degil" verdi.

Hocanın vurgusu: Bu çok kafa yormadan, "bilgisayar çalışsın, bellek çalışsın" denen naif bir çözümdür. Maalesef birçok yerde, bilgisayar mühendisliği bölümlerinde, hatta bazı kitaplarda çözüm olarak sunuluyor. Ama bir bilgisayar mühendisinden beklenen çözüm bu değildir, çünkü yeni bir belleğe ihtiyaç duyar.

7.2 Ek Dizi Kullanmayan Çözüm

`A[n-i-1]` ifadesini düşünebiliyorsak ekstra alana gerek yoktur. Baştan başlayıp 0. eleman ile `n-1`. eleman, 1. eleman ile `n-2`. eleman karşılaştırılır. Dizinin tam ortasına gelene kadar devam edilir. Hoca bunu bir kontrol listesine benzetti: Arada bir tane şartı bozan durum varsa çıkmamız gerekir.

Akış diyagramı (slaytta `A[i]==A[n-i-1] && i<N/2` şartlı bir `while` ve ardından bir `if` olarak çizildi):

1. Başla.
2. `n` ve `A` dizisini kullanıcıdan al.
3. `i = 0`.
4. `A[i] == A[n-i-1]` ve `i < n/2` ise `i = i + 1` yap, bu adımı tekrarla.
5. Şart bozulunca: `i < n/2` ise "palindrom değil" yaz, değilse "palindrom" yaz.
6. Dur.

Karar basittir: `i` ortadaki elemana ulaşmadıysa arada bir eşitsizlik bulunmuştur. Ortadaki elemana geldiysek dizi palindromdur. `i` burada en fazla $n/2$'ye eşit olur.

```c
#include <stdio.h>
#define MAX 100
int main() {
    int i, n;
    int A[MAX];

    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &A[i]);
    i = 0;
    while (A[i] == A[n - i - 1] && i < n / 2)
        i++;
    if (i < n / 2)
        printf("palindrom degil\n");
    else
        printf("palindrom\n");
    return 0;
}
```

Hoca bu sorunun kodunu bilerek yazmadı, bazı şeyleri öğrencilerin kendisinin koda dönüştürmesi gerektiğini söyledi. Yukarıdaki kod, slayttaki akış diyagramından ders sonrasında kuruldu ve `gcc -Wall -std=c89` ile derlenip 7.1'deki dizilerle aynı sonuçları verdi. Hoca, `n/2`'nin de yine tam sayı bölmesi olduğunu ve akış diyagramında aşağı yuvarlama işaretiyle gösterilmesi gerektiğini tekrar hatırlattı.

8. 6 Basamaklı Palindrom Sayılar

Soru şudur: 6 basamaklı palindrom sayıları ekrana yazdıralım ve sayısını bulalım.

> **Ders sonrası not:** Hoca başlangıcı önce "1 milyondan 999.999'a" dedi. Doğrusu, hemen sonra söylediği gibi 100.000'den 999.999'a kadardır.

8.1 Naif Çözüm: Her Sayıyı Denemek

İlk akla gelen yol, 100.000'den başlayıp her sayının basamaklarını ayırmaktır. Birinci ile sonuncu, ikinci ile sondan bir önceki, üçüncü ile dördüncü basamak birbirine eşit mi diye bakılır. Bir sayaç tutulur, buna büyük `C` denir. Çünkü küçük `c` zaten bir basamağın adıdır ve C büyük-küçük harfe duyarlıdır (case sensitive).

Basamaklar hoca tarafından şöyle gösterildi: `A = i / 100000`, en son basamak `F = i % 10`. Aradaki basamaklar da daha önce yapılan uygulamadaki gibi bölme ve mod işlemleriyle bulunur. Akış şöyledir:

1. `C = 0`.
2. `i` 100.000'den 999.999'a birer birer artan döngü aç.
3. `i`'nin altı basamağını `A, B, C, D, E, F` olarak bul.
4. `A == F` ise devam et.
5. `B == E` ise devam et.
6. Basamaklardaki `C == D` ise sayacı `1` artır.
7. Döngü bitince sayacı yaz ve dur.

8.2 Neden İç İçe `if`?

Bu üç kontrol birbirinden bağımsız `if`'lerle yazılmamalıdır. `A` ile `F` eşit değilse diğerlerine hiç bakılmamalıdır. Bağımsız yazılırsa eşit olmasa da bakmaya devam eder ve gereksiz işlem yapılır. Dahası üç `if`'in her birinde sayaç artarsa bu kontroller bir "veya" gibi çalışır. Üçünden birisi eşit olsa fazladan sayım yapılır ve yanlış sonuç bulunur.

Üç şart tek bir && ifadesinde de yazılabilir: `A == F && B == E && C == D`. Hoca bu yazımın da olabileceğini söyledi. Ama arada başka işlemler yapılması gereken durumlarda, örneğin A, F'ye eşitken A'yı bir artırıp F'yi bir azaltmak gerekiyorsa, iç içe `if` yazmak gerekir.

> **Hocanın vurgusu:** Bunlar birbirinden bağımsız `if`'ler değildir, dikkat edin. Öyle olursa gereksiz işlem yaptırmış, hatta yanlış sonuç bulmuş oluruz.

8.3 İşlem Sayısı

Soru: Burada en sık yapılan işlem hangisi, kaç defa yapılıyor? Öğrenciler önce karşılaştırma dedi. Hoca, karşılaştırmadan önce basamakları bulan matematiksel işlemlerin yapıldığını söyledi. 100.000 ile 999.999 arasında 900.000 sayı vardır. Her sayı için en az 6 temel işlem yapılır, atamaları da sayarak. Karşılaştırmalar hariç en az $6 \times 900\,000$ işlem eder.

Bir öğrenci iyileştirme önerdi: Önce yalnız `A` ve `F` bulunur. Eşitlerse `B` ve `E`, onlar da eşitse `C` ve `D` bulunur. Böylece `A` ile `F`'nin tutmadığı sayılarda diğer basamaklar hiç hesaplanmaz. İşlem sayısı $6 \times 900\,000$'den yaklaşık $2 \times 900\,000$'e düşer. Hoca bunu "şu kısmın bir kısmını buraya, bir kısmını buraya alalım" diye gösterdi.

```c
#include <stdio.h>
int main() {
    long i;
    int a, b, c, d, e, f;
    int C = 0;

    for (i = 100000; i < 1000000; i++) {
        a = i / 100000;
        f = i % 10;
        if (a == f) {
            b = (i / 10000) % 10;
            e = (i / 10) % 10;
            if (b == e) {
                c = (i / 1000) % 10;
                d = (i / 100) % 10;
                if (c == d)
                    C = C + 1;
            }
        }
    }
    printf("%d\n", C);
    return 0;
}
```

Hoca bu çözümü yalnızca akış diyagramı olarak verdi. Kod ders sonrasında kuruldu, `gcc -Wall -std=c89` ile derlendi ve 900 sonucunu verdi.

> **Ders sonrası not:** Hoca "2×900 bin mertebesinde, yaklaşık" dedi. Python ile sayıldı: 900.000 sayının hepsinde 2 atama, 90.000 sayıda 2 atama daha, 9.000 sayıda 2 atama daha yapılır. Toplam 1.998.000 atama olur, yani yaklaşık $2{,}22 \times 900\,000$.

8.4 Üretme Yaklaşımı: ABCCBA

Bir öğrenci farklı bir çözüm önerdi: Sayıyı kontrol etmek yerine doğrudan üretelim. 6 basamaklı palindrom `ABCCBA` biçimindedir. `A` en az 1 olmalıdır, çünkü sayı en az 100.000'dir. O hâlde:

1. `A` için 1'den 9'a bir döngü aç.

2. İçinde `B` için 0'dan 9'a bir döngü aç.
3. İçinde `C` için 0'dan 9'a bir döngü aç.
4. `sayi = A*100000 + B*10000 + C*1000 + C*100 + B*10 + A` hesapla, yazdır, sayacı artır.
5. Döngüleri içten dışa doğru kapat. Sayacı yaz ve dur.

Öğrenci formülü söylerken son terimlerde önce `B` ile `C`'yi karıştırdı, hoca ile birlikte `C*100 + B*10 + A` olarak düzeltildi.

Başka bir öğrenci, bir arkadaşının önerisini de aktararak aynı harfleri toplamanın işlem sayısını azaltacağını söyledi: `A*100001`, `B*10010`, `C*1100`. Hoca önce anlayamadı, sonra "bir adım daha kısalır" diye kabul etti.

```c
#include <stdio.h>
int main() {
    int a, b, c, t = 0;
    long sayi;

    for (a = 1; a <= 9; a++)
        for (b = 0; b <= 9; b++)
            for (c = 0; c <= 9; c++) {
                sayi = a * 100001L + b * 10010L + c * 1100L;
                printf("%ld\n", sayi);
                t = t + 1;
            }
    printf("%d\n", t);
    return 0;
}
```

Bu döngü kaç defa döner? Dış döngü 9, ortadaki 10, içteki 10 defa döner, çünkü bunlar sıfırdan başlar. Yaklaşık değil, tam $9 \times 10 \times 10 = 900$ defa döner. Yaklaşık $2 \times 900\,000$'den 900'e inilmiştir. Python ile doğrulandı: Üretilen 900 sayı, 100.000 ile 999.999 arasındaki palindromların tamamıdır ve sıralı üretilir. Kod da `gcc -Wall -std=c89` ile derlendi, 100001 ile başladı, 999999 ile bitti ve 900 saydı.

8.5 7 Basamaklı Palindromlar

Hoca aynı öğrenciye 7 basamaklı durumu sordu. Sayı `ABCDCBA` biçimindedir. Bir `D` eklenir ve 4 iç içe döngü olur:

$$\text{sayı} = 1\,000\,000 \cdot A + 100\,000 \cdot B + 10\,000 \cdot C + 1000 \cdot D + 100 \cdot C + 10 \cdot B + A$$

9. R Basamaklı Kaç Palindrom Sayı Vardır?

9.1 Yazdırmak ile Saymak Farklıdır

6 basamakta 3, 7 basamakta 4 döngü yettiğini gördük. Basamak sayısı kullanıcıdan `R` olarak alınırsa döngü sayısı R'ye bağlı olur ve iç içe döngü sayısını program yazarken belirleyemeyiz. Hoca bu yüzden iki durumu ayırdı.

- **Sayıların hepsini yazdırmak:** İç içe dinamik sayıda döngü yazılmaz. Tek bir döngüyle en küçük R basamaklı sayıdan en büyüğüne gidilir ve her sayı tek tek kontrol edilir. 8.1'deki çözüm budur.
- **Yalnızca sayısını bulmak:** Bu kadar dönmeye gerek yoktur. Düzeni bulup formül kullanılabilir.

  Ders sonrası not: Altyazıda hoca dinamik döngü için "iç içe R tane" ve 10 basamak için "10 çarpı 10 üzeri 10'dan başla" diyor. Hocanın kendi örneklerine göre palindrom üretmek için gereken döngü sayısı $\lceil R/2 \rceil$'dir (7 ve 8 basamakta 4). 10 basamaklı sayıların aralığı ise 10^9 ile $10^{10} - 1$ arasıdır. Hoca büyük olasılıkla bunu kastetmiştir.

9.2 Düzenin Bulunması

Bir öğrenci sayının $9 \times 10^{R/2}$ gibi olacağını söyledi, sonra yuvarlama yönü tartışıldı. Hoca "peki bunu nasıl ispat edeceğiz?" diye sorup küçük değerlerden başladı:

R	Yapı	Palindrom sayısı
1	Bütün sayılar (0 hariç)	9
2	11, 22, ..., 99	9
3	dış çift 9, araya 10 rakam	$9 \times 10 = 90$
4	dış çift 9, iç çift 10	$9 \times 10 = 90$
5	dış çift 9, iç çift 10, orta 10	$9 \times 10 \times 10 = 900$
6	dış çift 9, iki iç çift 10	$9 \times 10 \times 10 = 900$

Dıştaki basamak çifti 1'den 9'a kadar olduğu için 9 farklı değer alır. İçteki çiftler 0 da alabildiği için 10 farklı değer alır. Bir öğrenci önce 2 basamak için 90 dedi, hoca 18 ve 81 gibi sayıların palindrom olmadığını söyledi. Tablo ders sonrasında Python ile sayılarak doğrulandı.

Üstlere bakınca genel formül çıkar:

$$\text{R basamaklı palindrom sayısı} = 9 \times 10^{\lfloor (R-1)/2 \rfloor}$$

Hoca tabloyu yazarken üstlerde bir ara hata yaptı ($R = 5$ ve $R = 6$ için 3 yazdı). Bir öğrenci 900 olduğunu hatırlatınca üstleri 2 olarak düzeltti ve aşağı yuvarlamanın çalıştığını gördü.

9.3 Formülün Hesap Maliyeti

Formülü kodlamak için önce $t = \lfloor (R-1)/2 \rfloor$ bulunur. Bu birkaç işlemdir. Ama 10^t hesaplanmalıdır. Hazır bir `pow` fonksiyonu olsa da o da arka planda çarpma yapar.

Hocanın örnekleri:

- 20 basamaklı için: $19/2$'den $t = 9$ bulunur. 10^9 için döngü en az 9 defa döner. Sonuç 9×10^9'dur.
- 25 basamaklı için: $24/2 = 12$ bulunur, döngü 12 defa döner. Sonuç 9×10^{12}'dir.

Yani bu formülle de birim zamanda, 3-5 işlemde bulamayız, döngü $\lfloor (R-1)/2 \rfloor$ defa döner. Hoca, daha önce anlatılan yöntemi hatırlattı: A^{13} hesabındaki gibi ikili dizilim soldan sağa kullanılarak bu da log mertebesinde yapılabilir.

10. Uygulama Soruları

Hoca, asistan hocaların hazırladığı dört soruyu tek tek gösterdi. Çözülmesi gerekip gerekmediğini de öğrencilere sordu.

10.1 Tek Sayıların Çarpımı, Çift Sayıların Toplamı

1 ile 20 arasındaki tek sayıların çarpımı ve çift sayıların toplamı istenir. Sayı tekse çarpılır, çiftse toplanır. Toplam 0'dan, çarpım 1'den başlatılır. Hoca bunu BBG 1 seviyesinde bir soru olarak gördü ve yazmadı.

10.2 Girilen Sayıyı Tersine Çevirme

Hoca bu soruyu güzel buldu ve kodladı. Sayının tersini almak için önce birler basamağı alınır, en yüksek ağırlıklı basamak en sona kalır. Birler basamağı `% 10` ile alınır, sayı sıfıra inene kadar bu işlem tekrarlanır. Sayı bölündükçe küçüleceği için sayının kendisi bozulmasın diye önce yedek bir değişkene kopyalanır. `ters` ilk başta 0'dır, çünkü üzerine ekleme yapılacaktır.

```c
#include <stdio.h>
int main() {
    int temp, sayi;
    int ters = 0;

    scanf("%d", &sayi);
    temp = sayi;
    while (temp > 0) {
        ters = ters * 10 + temp % 10;
        temp /= 10;
    }
    printf("%d - %d", sayi, ters);
    return 0;
}
```

Kod ekrandan alındı, ders sonrasında derlendi. Hocanın 123 örneğinin adımları:

Tur	`temp % 10`	`ters`	`temp` (sonra)
1	3	$0 \times 10 + 3 = 3$	12
2	2	$3 \times 10 + 2 = 32$	1
3	1	$32 \times 10 + 1 = 321$	0

Derste 12345 için 54321, çift basamaklı 1234 için 4321 çıktı. Ders sonrası denemede de aynı sonuçlar alındı.

Kod yazılırken üç hata oluştu ve hoca bunları canlı düzeltti:

- `tersi` diye yazılmış değişken derleyiciden "undeclared" hatası aldı.
- `sayi /= 10` yazılınca döngü `temp`'e bağlı olduğu için `temp` azalmıyordu. `temp /= 10` olmalıydı.
- `sayi % 10` yazılınca çıktı, 12345 girişi için sürekli 5 ekleyerek yanlış geldi. Modu `temp`'ten almak gerekiyordu.

Program ayrıca çalıştığında hiçbir şey yazmadan bekledi. Hoca bunun sonsuz döngü olmadığını gösterdi: `printf` yazılmadığı için program sessizce `scanf`'te bekliyordu. Hocanın benzetmesi: ATM'ye gittiniz, karşınızda hiçbir mesaj yok, bekliyorsunuz. Sabaha kadar bekleyebilirsiniz.

10.3 Armstrong Sayıları

n basamaklı bir sayının basamaklarının n. kuvvetleri toplamı kendisine eşitse bu bir Armstrong sayısıdır. Yani `abcd...` için $a^n + b^n + c^n + \cdots$ sayının kendisine eşit olmalıdır. 3 basamaklıysa küpler, 4 basamaklıysa 4. kuvvetler alınır. Soru 100 ile 1000 arasındaki Armstrong sayılarını istiyor.

Bir öğrenci önceki örnekteki gibi `A`, `B`, `C` için üç döngüyle sayıyı üretmeyi önerdi: `A` 1'den, `B` ile `C` 0'dan başlar. Hazırda değişkenler olduğu için mod işlemine gerek kalmaz. Hoca bunun bu aralık için güzel ama spesifik bir çözüm olduğunu söyledi. Girilen bir sayıya kadar, örneğin 500 bine kadar denseydi bu çözüm yetmezdi.

> **Hocanın vurgusu:** Genel çözüme gideceğiz. Özel bir aralığa göre yazılmış çözüm ile genel çözüm arasındaki fark görülmelidir.

Genel çözüm şöyledir: Dış döngü sayıları dolaşır. İçteki `while` döngüsü, sayı sıfırdan farklı olduğu sürece önce modu alıp basamağı bulur, küpünü toplama ekler, sonra sayıyı 10'a böler. Hoca hazır çözümü ekrana kopyaladı:

```c
#include <stdio.h>
int main() {
    int i, sayi, toplam, basamak;

    for (i = 100; i <= 1000; i++) {
```

```
        sayi = i;
        toplam = 0;
        while (sayi != 0) {
            basamak = sayi % 10;
            toplam += basamak * basamak * basamak;
            sayi /= 10;
        }
        if (i == toplam)
            printf("%d\n", i);
    }
    return 0;
}
```

Soru: `toplam += ...` satırı ile `sayi /= 10` satırını yer değiştirsek sorun olur mu? **Cevap:** Olmaz, çünkü bu iki satırdaki değişkenler birbirini etkilemez. Önemli olan, `basamak` satırının `sayi /= 10`'dan önce gelmesidir. Ders sonrasında iki sıralama da derlendi, aynı sonucu verdi.

> **Ders sonrası not:** Hoca bu kod üzerinde konuşurken "90 tane sayı gelecek, palindrom olan" dedi. Bu cümle 9. bölümdeki 3 basamaklı palindrom sayısına aittir. Kodun çıktısı Armstrong sayılarıdır. Program çalıştırıldı, 153, 370, 371 ve 407 olmak üzere 4 sayı yazdırdı (Python ile de aynı). Kodun `if` satırı ve devamı karede görünmüyor, altyazıya göre tamamlandı.

Hoca, girilen bir sayıya kadar istense her sayının tek tek kontrol edileceğini, sayının basamak sayısının da buradan hesaplanabileceğini söyledi. Bir kelimede kaç harf olduğu ise gelecek derste konuşulacaktır.

10.4 Onluktan İkiliğe Çevirme

10 tabanında girilen bir sayının 2 tabanına çevrilmesi gelecek derse bırakıldı. Hoca isteyenlerin çalışmasını istedi.

11. Kod Standartları ve Kısıt Altında Çözüm

Hoca, öğrencilerin soru tarzını sorması üzerine düşünme biçimini anlattı. Doğrudan "dizinin en küçüğünü bul" sorusu bir ezberdir. Asıl mesele bunu transfer edebilmektir: notların arasından en küçüğünü, yaşların arasından en büyüğünü bulmak gibi.

Hocanın örneği şudur: Öğrencilerin vize ve final notları iki boyutlu bir matriste tutuluyor. Birinci sütunda vizeler, ikinci sütunda finaller var, üçüncü sütun yok. Ortalaması en büyük olan öğrenci, bu matrisi bozmadan ve ek bir dizi kullanmadan bulunacak. Yani çözüm o yapıyı bozmadan, o kısıtlar altında yapılacak. Matrisi 90 derece döndürme gibi sorular da ileride bu çerçevede gelecektir.

Bu dersin C kodu standartları da burada tekrarlandı:

- Yalnızca `main` fonksiyonu içinde, şu ana kadar görülen konularla kod yazılır. Fonksiyon kullanılmaz.
- `for (int i = ...)` gibi yazım kullanılmaz. Hocaya göre bu tür tanımlar dersin derleyicilerinde derlenmez.
- Döngülerde `break`, `continue` ve `goto` kullanılmaz.
- Sorudaki kısıtlar ihlal edilmez, örneğin istenmeyen ek dizi veya matris tanımlanmaz.

Hoca bunu dersin amacıyla bağladı: Bu dönem boyunca her soruda önce bir algoritma tasarlandı, sonra "daha iyisini yapalım" denildi. Matris kullanarak yapılan şey ilk akla gelen çözümdür. Kısıtlar, makul sürede daha iyisini bulmak için konur.

> **Hocanın vurgusu:** Bu ders, daha iyi algoritma yazmak üzerine kurulu bir derstir. Bu bir BBG dersi değil.

Soru: Akış diyagramında `for` içinde n yazınca, n dahil mi demek olur? **Cevap:** n dahil edilecekse önüne eşittir konur. Yani n'e kadar ama n dahil değil demek için küçüktür n, dahil demek için küçük eşit n yazılır.

12. Ders Sonrası Soru: Kosinüs Serisi

Ders bittikten sonra bir öğrenci ekranını paylaşarak kendi kodunu sordu. Derste hesaplanması istenen kosinüs serisini iki ayrı fonksiyonla (üs ve faktöriyel) yazmıştı:

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

Öğrencinin sorusu şuydu: e^x örneğinde olduğu gibi bir önceki terimin değerini kullanabilir miyiz? x^2'den x^4'e geçmeyi biliyorum, ama $2!$'den $4!$'e nasıl geçerim?

12.1 Önceki Terimden Sonrakine

Hoca kod üzerinden gösterdi. Döngüden önce `ustlu = 1`, `faktoriyel = 1` tutulur, bu $x^0/0!$ demektir. Sonuca `s * ustlu / faktoriyel` eklenir, `s` işaret değişkenidir. Sonra terimler güncellenir:

```
ustlu = ustlu * x * x;
faktoriyel = faktoriyel * (t - 1) * t;
t = t + 2;
```

`t` değişkeni 2'den başlar. Hoca adımları izledi:

`t`	Faktöriyel güncellemesi	Sonuç
2	$1 \times 1 \times 2$	$2! = 2$
4	$2 \times 3 \times 4$	$4! = 24$
6	$24 \times 5 \times 6$	$6! = 720$

Tam program (derste öğrencinin kodu üzerinde satır satır değiştirildi, bütünü görünmüyor; ders sonrasında kuruldu):

```c
#include <stdio.h>
#define PI 3.14159265358979
int main() {
    int k, terim, t = 2, s = 1;
    double x, derece, r = 0, ustlu = 1, faktoriyel = 1;

    scanf("%lf %d", &derece, &terim);
    x = derece * PI / 180;
    for (k = 0; k < terim; k++) {
        r = r + s * ustlu / faktoriyel;
        s = -s;
        ustlu = ustlu * x * x;
        faktoriyel = faktoriyel * (t - 1) * t;
        t = t + 2;
    }
    printf("%f\n", r);
    return 0;
}
```

Derste 60 derece ve 10 terim girildi, $\cos 60^\circ = 0{,}5$'e çok yakın sonuç bekleniyordu. Ders sonrasında kod `gcc -Wall -std=c89` ile derlendi, 60 ve 10 için 0.500000 verdi. Python ile de $0{,}5000000000000001$ çıktı. Hoca sonunda, bu şekilde yazınca öğrencinin iki fonksiyondaki döngülere gerek kalmadığını söyledi.

12.2 Hata Arama Süreci

Yöntemleri karşılaştırmak için hoca öğrenciye döngüyü kopyalatıp iki sürümü aynı anda çalıştırdı. Aynı sonucu vermeleri bekleniyordu, ama öğrencinin fonksiyonlu sürümü farklı çıktı. Bunun üzerine aşama aşama arama yapıldı.

- Faktöriyel fonksiyonu satır satır okundu ve doğru bulundu.
- Üs fonksiyonu tek başına denendi: 2 ve 5 argümanlarıyla çağrıldı, 32 beklenirken 25 geldi.
- Fonksiyon üssü önce ikilik tabana çeviriyordu. Hoca 13'ü ikiye bölerek ikiliğe çevirmeyi Paint'te öğrenciye yazdırdı: 13 → 6 kalan 1, 6 → 3 kalan 0, 3 → 1 kalan 1, 1 → 0 kalan 1 (ara adımlar eklendi), yani 1101. Lise yönteminde son bölüm olan 1'in de alındığını hatırlattı ve kodda son bölümün diziye yazılmadığından şüphelendi. Öğrenci `A[i] = ust` satırını ekledi.
- Dev-C++'ta debug denendi, açılmadı. Bunun yerine `printf` ile ara değerler yazdırıldı.
- Sonra öğrenci asıl durumu fark etti: Fonksiyon parametreleri önce üs, sonra taban alıyordu. Yani çağrı $5^2 = 25$'i hesaplıyordu ve sonuç doğruydu. Parametreler yer değiştirince 32 çıktı (Python ile $2^5 = 32$, $5^2 = 25$).
- Üs fonksiyonu doğru çıkınca hoca kopyalanan iki döngüye geri döndü ve eksik olanı buldu: İkinci döngüden önce sonuç değişkeni `r` sıfırlanmamıştı, yani ikinci döngü ilkinin sonucunun üzerine ekliyordu. `r` sıfırlanınca iki sürüm aynı sonucu verdi.

  Ders sonrası not: Hoca "kalan olarak gitmiyor, döngü dönmeyeceği için kalan olarak gidemezsin" dedi. Öğrenci ise son 1'in zaten kalan olarak gittiğini söylemişti. Python ile 13 için yukarıdaki bölme adımlarına bakılırsa döngü bölüm 0 olana kadar sürdüğünde en anlamlı bit de kalan olarak alınır. Kod görünmüyor, ama parametre sırası düzeltilince sonuç doğru çıktığına ve eklenen satırla çıktıda fazladan bir 0 göründüğüne göre, eksik bit büyük olasılıkla gerçek hata değildi. Asıl fark büyük olasılıkla `r`'nin sıfırlanmamasından kaynaklanıyordu.

Hoca son olarak, bu tarz soruların mail yerine ders sonunda sorulmasının daha iyi olduğunu söyledi, çünkü konu sıcağı sıcağına çözülür.

Dersin Toparlaması

Bu derste aynı düşünce farklı problemlerde tekrarlandı: Önce ilk akla gelen çözüm kurulur, sonra adım sayısı ve bellek kullanımı sorgulanır. Swap'ta bir bellek ile üç ek işlem arasındaki trade-off görüldü ve toplama, çarpma gibi `temp`'siz yöntemlerin taşma ve sıfıra bölme riskiyle doğruluğu bozabileceği anlaşıldı. Pisagor üçlüsünde tur sayısı yaklaşık 1 milyardan 124.750'ye indi. Floyd üçgeninde n^2 mertebesinden n'e, oradan sabit sayıda işleme inildi. Dizi ters çevirme ve palindrom sorularında $\lfloor n/2 \rfloor$ sınırı, C'de indislerin sıfırdan başlaması ve ek dizi kullanmamak öne çıktı. Palindrom sayılarda ise kontrol etmek yerine üretmek, işi yaklaşık 2×900 binden 900 tura indirdi ve R basamak için bir sayma formülü bulundu.

Hoca özel çözüm ile genel çözüm arasındaki farkı ve kısıtlara uyarak daha iyisini aramayı dersin ana beklentisi olarak gösterdi. Sonraki akademik bağlantılar şunlardır: Onluktan ikiliğe çevirme ve bir kelimedeki harf sayısı gelecek derste ele alınacak, sıralama algoritmaları, fonksiyonlar ve dinamik bellek de ileride görülecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.